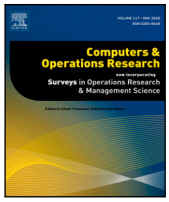
Computers & Operations Research 172 (2024) 106821



Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor





Redesigning a NSGA-II metaheuristic for the bi-objective Support Vector Machine with feature selection

Javier Alcaraz

Center of Operations Research (CIO), Miguel Hernández University (UMH), Spain

ARTICLE INFO

Keywords:
Support vector machine
Feature selection
Multi-objective optimization
Metaheuristics

ABSTRACT

The Support Vector Machine is a well-known technique used in supervised classification. Feature selection offers several benefits but also adds complexity to the problem. In this paper, we consider the soft margin SVM and given that two different objectives are considered simultaneously, obtaining the Pareto front , or at least a good approximation of it, gives the decision-maker a wide variety of solutions and has several advantages over having only one solution. The only metaheuristic that has been developed to give an approximation of such a front is a NSGA-II based technique. However, the design of such technique presents some limitations that are analyzed in this paper. We present a new metaheuristic that has been completely redesigned in order to overcome those drawbacks. We compare both techniques through an extensive computational experiment that demonstrates the superior efficiency of the new technique.

1. Introduction

The problem of how to classify different objects into groups or classes is a commonplace in our daily lives, being present in companies or organizations of very different sectors: in medicine the doctors want to know if a person has cancer or not; in image recognition, we want to determine if an image corresponds to a star or a planet; in banking, it is important to determine if a client will be able to pay their mortgage or not; in botany, we want to know with the data available, which species a plant specimen belongs to, etc. Several procedures have been designed to carry out a supervised classification and the Support Vector Machine (SVM) has become an efficient tool to carry out this task.

SVM was first introduced by Vapnik (1995) and Cortes and Vapnik (1995) as a way of minimizing the confidence interval while keeping the value of the empirical risk invariant. If we have only two classes of objects (commonly referred to as vectors) to be classified, the SVM calculates a separating hyperplane that divides the space into two regions, leaving the vectors of each class in one of the regions. To build this hyperplane, first two parallel hyperplanes, each one of them supported by some vectors of each class are formed and the distance between them is referred to as the margin. Then, the classifier is the intermediate parallel hyperplane. If misclassified vectors are allowed, either to achieve better generality or because data are not linearly separable, the margin is said to be soft. Then, in the soft margin SVM we look for solutions with a large margin and a small distance between the misclassified vectors and the corresponding parallel hyperplane. There are hundreds of areas where the SVM has been successfully applied, to cite only a few: in medicine, for the diagnosis of cancer (Ding et al., 2023), for the prediction of mortality (Amini et al., 2023) or survival in intensive care (Luaces et al., 2009); in biology for clinical classification (Tay et al., 2014); in image recognition and classification (Zhang and Chow, 2012; Gu et al., 2017); in botany for classifying biotopes (Middleton et al., 2012) and in banking for fraud detection (Pai et al., 2011). A recent survey on SVM, including applications and trends, was carried out by Cervantes et al. (2020).

SVM models can present an important drawback: the overfitting to the training data. To avoid this, several authors have proposed mechanisms to make a selection of the most discriminative features. For example, Maldonado et al. (2014) propose two different mixed integer linear programming formulations that obtain better predictions with consistently fewer features; Aytug (2015) develops an exact method based on generalized Benders decomposition and they also propose a relaxation of the problem where selecting too many features is penalized; Gaudioso et al. (2017) propose a Lagrangian-relaxation-based heuristic, embedding a weighted combination of the $L1$ and $L0$ norm of the normal to the separating hyperplane; Huang et al. (2018) propose an enhanced version of the recursive feature elimination algorithm, based on feature clustering, that reduces the computational complexity of the original method; Labbé et al. (2019) propose a mixed integer linear programming formulation where the choice of suitable features is modeled by introducing a budget constraint. Feature selection also allows operationality to be added by reducing the size in problems with vectors with a large amount of characteristics or features.

E-mail address: jalcaraz@umh.es.

https://doi.org/10.1016/j.cor.2024.106821
Received 9 November 2023; Received in revised form 21 March 2024; Accepted 22 August 2024
Available online 24 August 2024
0305-0548/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

Feature selection is not restricted to SVM models but is employed in other classification techniques to maintain or even improve the classification accuracy. Several works deal with feature selection in a multi-objective scenario. To cite a few recent ones, Jiao et al. (2022) analyze feature selection by considering the minimization of the number of selected features and the maximization of the classification performance simultaneously but in a separate way; Wei et al. (2023) and Li et al. (2023) propose evolutionary algorithms for feature selection in high-dimensional classification, demonstrating their good performance when compared with several state-of-the-art algorithms after solving some real-world high-dimensional datasets; Wang et al. (2023) propose a multi-objective approach to efficiently handle the trade-off between diversity and convergence of the solutions. The experiments demonstrate that the method outperforms the current popular multi-objective feature selection methods. A good survey about multi-objective feature selection can be found in Al-Tashi et al. (2020).

Given the multi-objective nature of the soft margin SVM, several metaheuristics have been proposed. However, most of the methods developed combine both objectives in a unique objective function, introducing one or more non-problem parameters or hyperparameters. In these cases, the techniques designed can optimize the values of these parameters and the subset of features for solving the single-objective model. To cite some, Huang and Wang (2006), Zhao et al. (2011) and Gauthama Raman et al. (2017) propose genetic algorithms or hybridized variants to optimize the selection of features and the hyperparameters of the model; Lin et al. (2008b) develop a particle swarm optimization based technique and Lin et al. (2008a) a simulated annealing algorithm for the same purpose; García-Pedrajas et al. (2014) develop a memetic technique for simultaneous instance and feature selection; Carrizosa et al. (2014) propose a heuristic to optimize the hyperparameters of most known SVM models; Aladeemy et al. (2017), Faris et al. (2018) also propose heuristics to optimize hyperparameters and a set of features simultaneously; Bouraoui et al. (2018) develop a multi-objective metaheuristic to optimize the subset of features and hyperparameters of the model, combining different objectives, such as accuracy, number of support vectors, margin and number of features; Candelieri et al. (2019) propose the use of parallel optimization for tuning the hyperparameters of the SVM models; Dudzik et al. (2021) propose an evolutionary technique for optimization, encompassing the training sample, the hyperparameters and the set of features. The methods mentioned above have, in general, an important drawback; there are a large number of different SVM models that include different hyperparameters and therefore, the election of the most appropriate one is itself an optimization problem. A model could perform well for a given instance or a set of these and very poorly in another set. Moreover, the efficiency of the SVM highly depends on the hyperparameter setting and the subset of features.

In this work, we deal with the soft margin SVM with feature selection, where the number of features is given. As misclassified vectors are allowed, the problem considers the optimization of two objectives: the maximization of the distance between the parallel hyperplanes and the minimization of the sum of the distances of the misclassified vectors to their corresponding hyperplane. The first objective represents the structural risk and the second the empirical risk. Therefore, the model to find the classifier becomes a bi-objective problem. Consequently, the best classifier is no longer a unique solution, but a set of non-dominated solutions that make up the so-called Pareto front. Moreover, setting the number of features acts as a budget constraint and makes the problem harder. The introduction of such constraint makes the problem non-convex and therefore obtaining the Pareto front with a single-objective function is not possible (Alcaraz et al., 2022). Therefore, the techniques cited in the previous paragraph fail in the purpose of obtaining the Pareto front , or a good approximation, for a soft margin SVM with feature selection, which is the objective of the technique presented in this work, since they only optimize one objective, which is a compromise between the two mentioned above. Obtaining the Pareto front of a multi-objective problem poses a challenge from a mathematical point of view, but in practice, also offers a decision-maker a rich set of alternatives from which a better decision could certainly be made. Having this set of solutions has several practical advantages. For instance, it enables the robustness of the solutions to be evaluated across various scenarios, thereby enabling the decision-maker to elaborate reasoned reports on the solutions to choose based on them. Moreover, a single solution may be highly sensitive to the parameters of the problem and could become suboptimal with small changes in these parameters, whereas having a set of solutions reduces the risk of this happening. Finally, the diversity of the solutions provides insight into different aspects of the problem, revealing unexpected connections between objectives. The cited work also demonstrates that obtaining the non-dominated solutions for the soft margin SVM with feature selection through exact techniques is unsuitable for even small-sized instances and therefore the authors develop the unique method, as far as we know, that is capable of generating the Pareto front, or at least a good approximation, in an efficient way without limitations in the size of the problem. Although the computational experiment carried out in that work demonstrates the efficiency of the method, we set out to demonstrate in this work that the design of the metaheuristic does have some significant drawbacks which we analyze in depth later: (i) the search space where the algorithm looks for solutions excludes some promising solutions; (ii) some instances of the problem cannot be solved and (iii) the computation times needed to solve some instances is extremely high. To avoid this, we present a metaheuristic that overcomes those drawbacks and outperforms the first. Although the new metaheuristic is based on the same template as the revised algorithm, it is completely new. A new way to encode the solutions that combine a real and an integer part has been designed. Now, the reduction of the search space, that eliminated a lot of good solutions as we will discuss later, is not carried out. In addition, this innovative encoding permits to solve any instance of the problem and the computation times are reduced very noticeably. Moreover, new procedures to combine and generate solutions have been designed, which makes the new algorithm innovative and efficient. The computational experiments carried out show that the algorithm proposed in this work is the best algorithm in the literature for generating a good approximation of the Pareto front when dealing with the soft margin SVM with feature selection.

This paper is organized as follows: Section 2 formally defines the problem considered and Section 3 describes the metaheuristic we have outperformed. The new metaheuristic proposed is presented in detail in Section 4 and the computational experiment comparing both techniques is presented in Section 5. Finally, Section 6 draws the conclusions and makes some final remarks.

2. The soft margin SVM with feature selection

When we have a set of m vectors Ω to be classified into just two classes, the vectors can be defined by a pair $(x_i, y_i) \in \mathbb{R}^n \times \{-1, 1\}$, where n is the number of characteristics that each one presents: the first element contains the feature values for the corresponding vector and the second defines the class of the given vector. The soft margin support vector machine consists in determining the hyperplane $f(x) = w^T \cdot x + b$ that separates the vectors in such a way that the distance between two parallel hyperplanes, supported by some vectors of both classes is maximized and the sum of classification errors is minimized. The soft margin SVM model presented in Bradley and Mangasarian (1998) minimizes a compromise between the above two objectives in a unique objective function and does not include feature selection. The model only uses continuous variables and is easy to solve. However, incorporating feature selection necessitates the introduction of binary variables and therefore the model becomes harder and non-convex. The new model, considering both objectives separately can be written as follows:

$$
\begin{aligned}
&\min && \|w\|^2 && \\
&\min && \sum_{i=1}^{m} \xi_i && \\
&\text{s.t.:} && && \\
&y_i(w^T x_i + b) \geq 1 - \xi_i && && i = 1, \ldots, m \\
&\sum_j t_j = p && && \\
&\xi_i \geq 0 && && i = 1, \ldots, m \\
&|w_j| \leq M t_j && && j = 1, \ldots, n \\
&w_j \in \mathbb{R} && && j = 1, \ldots, n \\
&b \in \mathbb{R} && && \\
&t_j \in \{0, 1\} && && j = 1, \ldots, n
\end{aligned}
$$

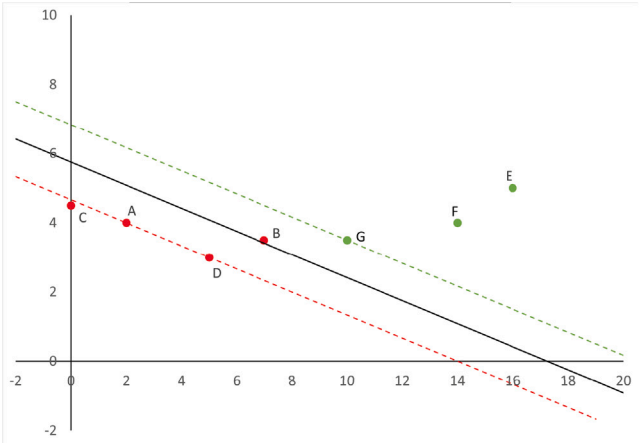
Continuous variables w_j and b set the coefficients of the two parallel hyperplanes $w^T x + b = 1$ and $w^T x + b = -1$. The family of binary variables t_j are used for selecting ($t_j = 1$) or not selecting ($t_j = 0$) feature j. The first objective function, to be minimized, represents the structural risk, given that $\|w\|$ is twice the inverse of the distance between the two parallel hyperplanes. Therefore, minimizing $\|w\|^2$ is equivalent to maximizing the distance between both hyperplanes, D. Details about this equivalence can be found in the work by Alcaraz et al. (2022). The second objective, the sum of the deviation of misclassified vectors, represents the empirical risk, and is to be minimized.

Example 1. Let $\Omega = \{A, B, C, D, E, F, G\}$ be the set of the 7 vectors ($m = 7$) represented in Table 1. As we can observe, vectors are divided into two different classes, *Class 1* with 4 vectors and *Class 2* with 3. Vectors are defined by three different features or dimensions ($n = 3$), 1, 2 and 3. If, for example, $p = 2$ is set, only two of the three dimensions must be selected. The problem consists of building two parallel hyperplanes, one of them supported by, at least one vector of *Class 1* and the other supported by, at least, one vector of *Class 2*. The distance between both hyperplanes, D, is to be maximized. Then, the intermediate parallel hyperplane is the corresponding classifier. The sum of the distances of the misclassified vectors to their corresponding hyperplane, ε, is the second objective, to be minimized.

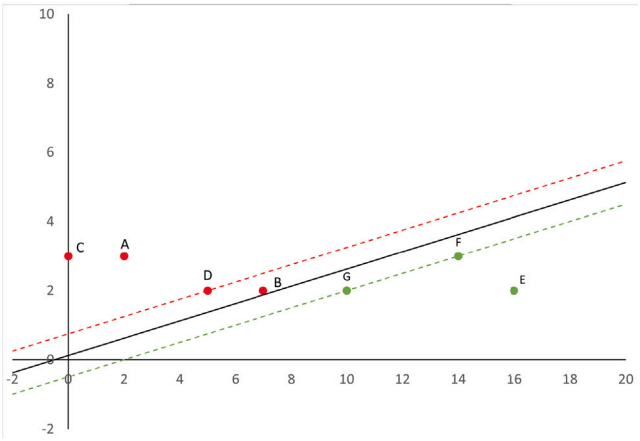
Figs. 1 and 2 show 4 different solutions to the dataset presented in Table 1. In these figures, the parallel hyperplanes, supported by vectors of the corresponding classes are drawn with dashed lines and the intermediate parallel hyperplane with a black non dashed-line. In Solution 1(b), the two features chosen are 1 and 3. In solutions 1(a), 2(a) and 2(b), the selected features are 1 and 2. In solution 1(a), the hyperplanes are supported by two vectors of *Class 1* and one vector of *Class 2* and in Solution 1(b) the corresponding hyperplanes are supported by one vector of *Class 1* and two vectors of Class 2. In Solutions 2(a) and 2(b), the hyperplanes are supported by only one vector of each class. It is easy to calculate the values for both objectives, D and ε, for these four solutions, which are represented in Table 2. As we are dealing with the problem from a multi-objective perspective, the concept of optimal solutions is replaced by the concept of non-dominated solutions. In this case, solutions 2(a) and 2(b), would not be dominated by any other of the solutions presented. By contrast, solution 1(a) is dominated by solution 2(a), given that the latter has better values in both objectives and solution 2(b) dominates solution 1(b). The set of all non-dominated solutions for a given problem forms the optimal Pareto front and the purpose is to achieve its best possible approximation.

Table 1
Example dataset with 7 vectors of 2 classes and 3 dimensions.

Vector	Class	1	2	3
A	1	2	4	3
B	1	7	3.5	2
C	1	0	4.5	3
D	1	5	3	2
E	2	16	5	2
F	2	14	4	3
G	2	10	3.5	2



(a) Features: 1, 2. The red hyperplane is supported by two vectors (A, D) and the green one is supported by one vector (G).



(b) Features: 1, 3. The red hyperplane is supported by one vector (D) and the green one is supported by two vectors (F, G).

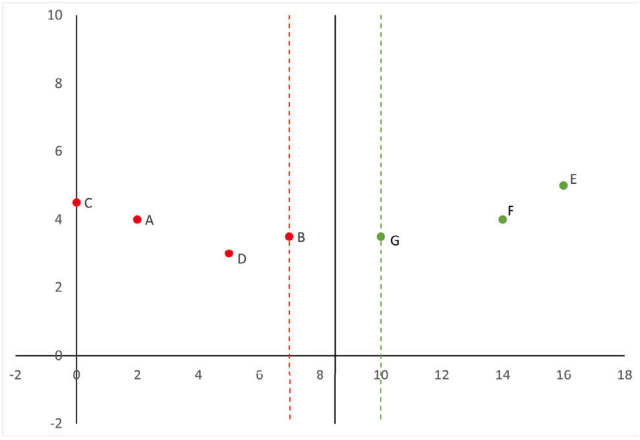
Fig. 1. Two different solutions for the problem presented in Table 1. One of the hyperplanes is supported by two vectors and the other by just one vector.

Table 2
Objective values for the solutions represented in Figs. 1 and 2.

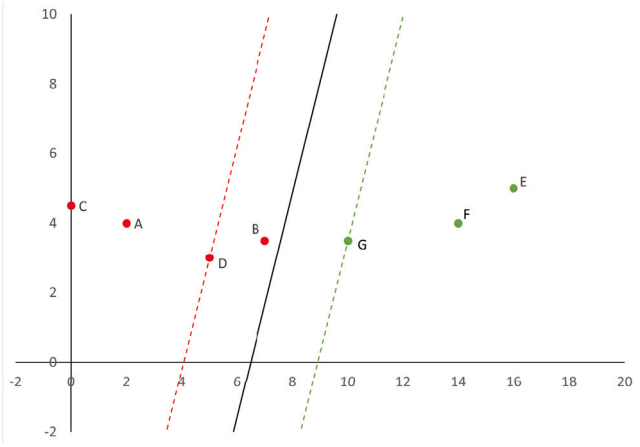
Solution	D	ε
Fig. 1(a)	2.0554	1.1068
Fig. 1(b)	3.1305	1.7688
Fig. 2(a)	3.0000	0.0000
Fig. 2(b)	4.6261	1.7614

3. A metaheuristic from the literature for the soft margin SVM with feature selection

The work by Alcaraz et al. (2022) is, as far as we know, the only algorithm developed to draw an approximation of the Pareto front of a soft margin support vector machine with feature selection. The algorithm, which we will refer to as Algorithm 1 from now on, follows the basic template of the general purpose multi-objective technique NSGA-II (Deb et al., 2002). A detailed description of the algorithm can be found in the referenced work although their main aspects will be discussed later.



(a) Features: 1, 2. The red hyperplane is supported by one vector (B) and the green one is also supported by one vector (G).



(b) Features: 1, 2. The red hyperplane is supported by one vector (D) and the green one is also supported by one vector (G).

Fig. 2. Two different solutions for the problem presented in Table 1. Both hyperplanes are supported by only one vector.

The algorithm demonstrates its efficiency for giving the set of non-dominated solutions when compared with an exact method as AUGMECON2 (Mavrotas and Florios, 2013) for solving different well-known datasets. They used ten different datasets. Six of them can be found in the UCI Machine Learning Repository (Kelly et al., 1987): Housing, GC, WBC, Ionosphere, Arrhythmia and Mfeat. This repository is a collection of databases, domain theories, and data generators that are used by the machine learning community for the empirical analysis of machine learning algorithms. Four additional datasets, Colon, DLBCL, Leukemia and Carcinoma are also well-known instances previously used in several works and a further description of them can be found, for example, in Maldonado et al. (2014), Carrizosa et al. (2009), Guyon et al. (2002), Golub et al. (1999), Notterman et al. (2001) and Shipp et al. (2002). This set of ten instances, whose main characteristics are shown in Table 3, are grouped into two subsets, depending on the number of features available in the data: small (the number of available features varies between 13 and 33) and large (with a number of dimensions between 258 and 7457). In the small datasets, the exact method works reasonably well and the metaheuristic gives good approximations in a considerably smaller computation time. In the largest instances, the exact method needs very high computation times to obtain only some points of the frontier whereas the metaheuristic obtains, in much smaller times, much better frontiers with a high number of points. However, in the computational experiment carried out in the work, all the instances have been solved with a fixed

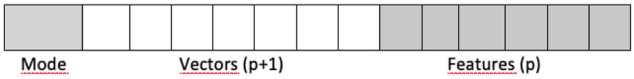
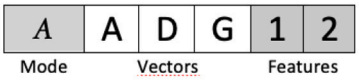


Fig. 3. Encoding for a solution with selection of p features.



(a) Solution with mode A



(b) Solution with mode B

Fig. 4. Encoding for two different solutions of Example 1.

value of $p = 5$, that is, the method must choose only 5 features among the total.

The algorithm proposed is based on a encoding scheme which represents three items: the support vectors, the set of features chosen and the mode of the solution, which indicates the way in which the vectors are used to build the hyperplanes. The encoding scheme employed is represented in Fig. 3.

The set of the p features chosen is represented in the last part of the solution, where each one of the p genes represents one of the selected features. To build the corresponding hyperplanes, $p+1$ vectors need to be chosen: p of one class and only one of the other class. The p vectors allow the hyperplane supported by those vectors to be built. Then, the second hyperplane is built parallel to the first and supported by the other vector. There are only two possible modes to choose the vectors: mode A indicates that the first p vectors in the solution belong to *Class 1* and the last vector to *Class 2*; in mode B, only the first vector comes from *Class 1* and the following p from *Class 2*. In Fig. 4 two different solutions for the toy example of Table 1 have been presented. In the first, the selected features are 1 and 2 and in the second, 1 and 3. In the first solution, mode A indicates that the first hyperplane has to be built supported by the two vectors of *Class 1*, A and D, and then, the second hyperplane can be built parallel to the first and supported by vector G, belonging to *Class 2*. The second solution, with mode B, builds one hyperplane supported by two vectors of *Class 2*, F and G and then, a parallel hyperplane supported by vector D that belongs to *Class 1*. These two solutions correspond with those presented in Fig. 2. More details about the encoding and the algorithm can be found in the referenced work.

This encoding scheme, as we will discuss later, although representing an innovative way of encoding the solutions to the problem, also imposes a series of restrictions which makes the algorithm inappropriate for solving the problem. We think that the algorithm presents three main drawbacks:

- **Reduced search space.** Given that the encoding employed imposes the condition that one of the hyperplanes must be supported by at least p vectors of the corresponding class, the search space of the algorithm excludes several solutions to the problem. The algorithm reduces the search space eliminating all the solutions in which both hyperplanes are supported by less than p vectors. Sometimes, reducing the search space may be beneficial for an algorithm, facilitating the convergence and resulting in a better performance, if it follows from an analysis of the search space that ensures that the regions of the search space excluded do not contain valuable solutions. However, this is not the case for

Table 3
Limitation of maximum number of features (p) to be considered by Algorithm 1.

Instance	m	m_1	m_2	n	Max p
Housing	506	249	257	13	n
GC	1000	700	300	24	n
WBC	569	357	212	30	n
Ionosphere	351	126	225	33	n
Arrhythmia	420	183	237	259	237
Mfeat	2000	1800	200	649	n
Colon	62	40	22	2000	40
Leukemia	72	38	34	5327	38
DLBCL	77	19	58	7129	58
Carcinoma	36	17	19	7457	19

Algorithm 1. The reduction of the search is imposed by limitations of the encoding used, which excludes, a priori, many solutions that, as we will demonstrate later, can be part of the Pareto front. In this case, the search space is formed by a finite given number of solutions, N_S, which depends on the combinations of vectors and features that the encoding needs to represent one solution and is given in the following formula, where m_1 and m_2 represent the number of vectors of both classes, n the total number of features and p the number of features to be selected.

$$N_S = \binom{n}{p} [m_1 \binom{m_2}{2} + m_2 \binom{m_1}{2}]$$

For instance, in Example 1, the search space of Algorithm 1 would exclude the two solutions in Fig. 2. Therefore, it could exclude some or many non-dominated solutions making it impossible to obtain the optimal Pareto front or it could affect to the quality of the approximation frontier given by the algorithm. The search space of this algorithm is made up of a finite number of solutions, which represents a subset of the search space of the problem, and therefore, the number of non-dominated solutions it can give is also finite. To be more precise, the search space of the algorithm for this example would be made up of only 90 solutions whereas the search space of the problem is formed by an infinite number of solutions.

- **Some instances of the problem cannot be solved.** Given that the encoding designed for Algorithm 1 imposes the condition that one of the hyperplanes must be supported by at least p vectors, the algorithm could not solve instances where $p > max(m_1, m_2)$. If we consider the instances solved in the computational experiment carried out by the authors, we realize that this condition represents a significant drawback regarding the maximum value of p to be considered, as evidenced in Table 3. For example, in the Carcinoma dataset, although there are a total of 7457 dimensions, this algorithm could only solve the problem with a maximum value of $p = 19$ and problems with values of p varying from 20 to 7457 would be unsolvable. In the table, we can observe the large differences between the "Max p" value represented in the last column and the number of dimensions. In only 5 of the 10 datasets could Algorithm 1 solve all the possible cases.

- **Computation times.** To evaluate one solution, this algorithm builds the two parallel hyperplanes, one of them supported by p vectors of one of the classes and the second hyperplane, parallel to the first and supported by one vector of the other class. The time needed to build such hyperplanes and evaluate one solution dramatically increases as p increases. This is not evidenced in the computational experiment presented in the work since all the instances were solved with a unique value of $p = 5$. However, the computation times needed to evaluate one solution drastically increase with larger values of p. To show this, we have solved the instances presented in their work with Algorithm 1 but varying, as much as possible, the value of p. The time needed to evaluate one solution for each instance, depending on p, is shown in seconds in Table 4. In this table, "–" indicates that value for p is not allowed in that instance (it is larger than the number of dimensions). In the case that the algorithm cannot solve the instance due to the aforementioned drawback, it is presented as "unsolvable". If we observe the times needed to evaluate one solution, they are quite small in all the instances and, at around 0.00002 s, on average. For small values of p, the computation times increase at the same rate as p. For instance, the average computation time needed to evaluate one solution when $p = 20$ is approximately four times greater than when $p = 5$. But this rate is not maintained if p goes on growing. When p changes from 50 to 100 the computation time is multiplied by 10. In the case of Mfeat the computation time is multiplied by almost 50 when p changes from 200 to 500. This table also evidences the large number of cases where this algorithm cannot solve the instance given the restrictions of the encoding employed. The average times needed by Algorithm 1 to compute one solution have been represented in Fig. 5 where we can observe how the computation times increase as p increases.

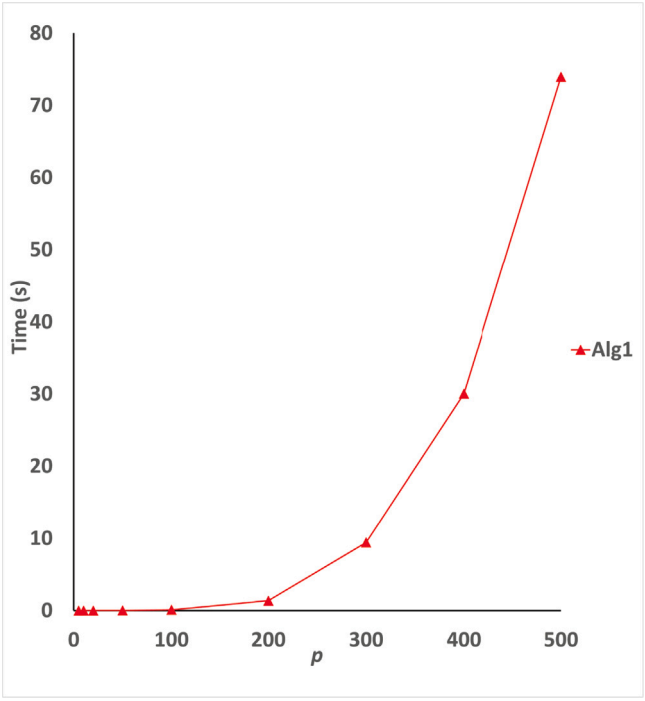


Fig. 5. Average computation times (s) of Algorithm 1 when varying p between 5 and 500.

To overcome the drawbacks presented above, we propose a new evolutionary algorithm that, although based on the same general NSGA-II template, makes use of a new encoding that, together with the specific genetic operators designed to work with it, makes the algorithm much more effective and efficient than the one presented in this section, as will be demonstrated in the extensive computational experiment carried out and presented in the corresponding section.

4. A new and competitive metaheuristic for the soft margin SVM with feature selection

We have completely redesigned the algorithm proposed by Alcaraz et al. (2022) in order to overcome the limitations that this algorithm presents and that have been discussed in the previous section. The redesign of the algorithm consists of proposing a new encoding for the

Table 4
Computation times (s) to evaluate one solution for Algorithm 1.

Instance	n	p=5	p=10	p=20	p=50	p=100
Housing	13	1.5E−04	3.0E−04	–	–	–
GC	24	1.3E−04	2.8E−04	6.6E−04	–	–
WBC	30	2.4E−04	4.0E−04	6.9E−04	–	–
Ionosphere	33	1.1E−04	2.5E−04	6.3E−04	–	–
Arrhythmia	259	5.3E−05	1.3E−04	2.6E−04	5.6E−03	8.2E−02
Mfeat	649	1.0E−03	1.7E−03	3.1E−03	1.5E−02	1.1E−01
Colon	2000	5.6E−05	1.1E−04	5.7E−04	unsolvable	unsolvable
Leukemia	5327	7.7E−05	1.6E−04	5.9E−04	unsolvable	unsolvable
DLBCL	7129	1.1E−04	1.5E−04	5.7E−04	8.6E−03	unsolvable
Carcinoma	7457	4.4E−05	1.1E−04	unsolvable	unsolvable	unsolvable
Average		2.0E−04	3.5E−04	8.9E−04	9.6E−03	9.8E−02

Instance	n	p=200	p=300	p=400	p=500	p=n
Housing	13	–	–	–	–	4.1E−04
GC	24	–	–	–	–	9.0E−04
WBC	30	–	–	–	–	1.1E−03
Ionosphere	33	–	–	–	–	2.3E−03
Arrhythmia	259	1.2E+00	–	–	–	unsolvable
Mfeat	649	1.5E+00	9.4E+00	3.0E+01	7.4E+01	2.1E+02
Colon	2000	unsolvable	unsolvable	unsolvable	unsolvable	unsolvable
Leukemia	5327	unsolvable	unsolvable	unsolvable	unsolvable	unsolvable
DLBCL	7129	unsolvable	unsolvable	unsolvable	unsolvable	unsolvable
Carcinoma	7457	unsolvable	unsolvable	unsolvable	unsolvable	unsolvable
Average		1.4E+00	9.4E+00	3.0E+01	7.4E+01	4.3E+01

solutions, which implies the design of new procedures and operators. The new algorithm, which we will hereinafter refer to as Algorithm 2 is presented, in pseudocode, in Algorithm 2 and as a flowchart in Fig. 6. Although both algorithms follow the basic scheme of the NSGA-II template, they are completely different. The new one employs an innovative encoding scheme and the different mechanisms, such as crossover and mutation, have been properly designed to manage and properly combine the information it contains to efficiently search for good solutions.

The first step consists in creating the initial population *(line 1)*, P_0 that, in this case is done by building N random solutions which are then evaluated *(line 3)*, i.e., both objectives, D and ϵ are computed for each solution. Before starting the main loop, the solutions are grouped into different fronts, depending on the solutions that dominate them *(line 4, fast_non_dominated_sort)*. F_1 is made up of the non-dominated solutions of the entire population, in F_2 we can find the solutions which are dominated by one or more solutions in F_1 and so on, that is, solutions in front F_i are dominated by one or more solutions placed in a front F_j for $j < i$. The main loop *(lines 5–26)* represents the evolution process, which is repeated until the stopping criterion is satisfied. First, a new population Q_t is created by repeatedly applying crossover and mutation operations to the solutions in P_t *(lines 6–13)*, and then the new solutions in Q_t are also evaluated *(line 14)*. R_t is a double-sized population, created by joining populations P_t and Q_t *(line 15)*. After this, solutions in R_t are grouped again in fronts *(line 16)*. Finally, to form the new current population, P_{t+1}, we must select the N best individuals in R_t. Therefore, individuals are selected in order, depending on the front they belong to. First, all the fronts that can be completely copied in the new population, are copied, in order *(lines 19–21)*. When all the individuals in the next front to be copied do not fit in the new population (condition in *line 19* is not satisfied, given that $|P_{t+1}|$ represents the size of the population) and it is not complete (N individuals), a distance criterion is used to select the best of the front. First, the distance between each of the solutions of the front to the rest of the solutions in it is calculated *(line 23, i_distance_assignment)*. Then, the solutions are sorted, in decreasing order by this distance *(line 24)* and finally, all the individuals in this front that fit in the new population, are copied in order *(line 25)*. This is the basic functioning of the algorithm, which follows the NSGA-II template exactly. For a detailed description of this, readers are referred to the NSGA-II original work (Deb et al., 2002).

Algorithm 2: New NSGA-II for SVM-f.

```
1  P_0 = create_initial_population(N);
2  t = 0;
3  P_0 = evaluate_population(P_0);
4  F = fast_non_dominated_sort(P_0);
5  while not stopping_criterion do
6      j = 1;
7      while j < N do
8          (parent1,
            parent2)=tournament_dominance_selection(P_t);
9          (Q_t[j], Q_t[j + 1])=crossover(parent1, parent2);
10         for k = 1 to 2 do
11             if random() ≤ PMUT then
12                 Q_t[j]=mutation(Q_t[j]);
13             j = j + 1;
14     Q_t = evaluate_population(Q_t);
15     R_t = P_t ∪ Q_t;
16     F = fast_non_dominated_sort(R_t);
17     P_{t+1} = ∅;
18     i = 1;
19     while |P_{t+1}| + |F_i| ≤ N do
20         P_{t+1} = P_{t+1} ∪ F_i;
21         i = i + 1;
22     if |P_{t+1}| < N then
23         i_distance_assignment(F_i) ;
24         sort(F_i, i_distance);
25         P_{t+1} = P_{t+1} ∪ F_i[1 : (N − |P_{t+1}|)];
26     t = t + 1;
27 return F_1
```

However, what makes an NSGA-II based algorithm different from any other, when applied to a given problem is, apart from the encoding (which is a crucial aspect to be considered), the way a new population Q_t is generated from the current population P_t *(lines 7–14)*. Each iteration of this loop creates two different solutions, that are placed in the new population, Q_t. First, two solutions from P_t,

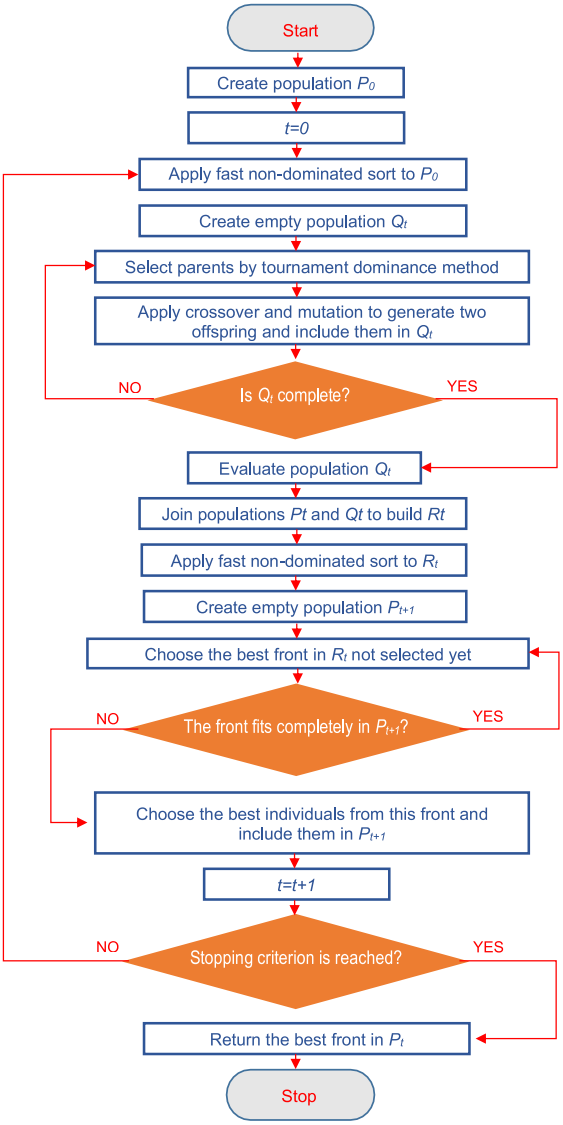


Fig. 6. Flowchart for the new algorithm (Algorithm 2).

the parents, are selected by a tournament-based procedure, considering dominance characteristics for deciding the winner *(line 8, tournament_dominance_selection)*. Then, they undergo the crossover operation to produce two offspring *(line 9)*, that are copied in Q_t in consecutive positions (j and $j+1$). Finally, these solutions undergo the mutation procedure *(lines 10–13)* with a given probability, that can slightly alter the solution. These mechanisms, together with the main features of the new algorithm are explained in detail in the following subsections.

4.1. Encoding

As we discussed previously, one of the main drawbacks of Algorithm 1, imposed by the design of the encoding, was the need that one of the hyperplanes that represents the solution is supported by p vectors while the other is supported by just one vector. This obligation reduces the search space that could not include some or all the non-dominated solutions of the problem. The new encoding eliminates this need and now only one support vector per class is imposed. With only two vectors, two parallel hyperplanes are not defined, so the coordinates of such hyperplanes need to be set. These p coordinates are real numbers. Once we have chosen both vectors and the coordinates have been

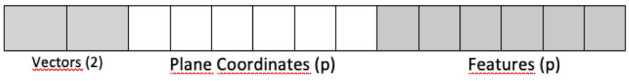


Fig. 7. New encoding for a solution with selection of p features.

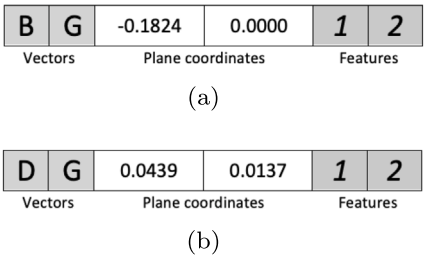


Fig. 8. New encoding for two different solutions of Example 1.

set, the two parallel hyperplanes can be built and the solution can be evaluated, i.e, both objectives (distance and epsilon) can be computed. Now, the search space does not exclude any solution.

Fig. 7 shows the new encoding for a problem with p features to be chosen. First, two vectors, one from each class, need to be chosen. Then, p real values indicate the coordinates of the hyperplane and finally, as in the previous encoding, the p features chosen in this solution are encoded. Fig. 8 shows two different solutions, with the new encoding, for the problem presented in Example 1. Solutions in Fig. 8(a) and (b) would correspond with those presented in Fig. 2(a) and (b). These two solutions would not be part of the search space of Algorithm 1, given that the hyperplanes are supported by only one vector. Therefore, as we discussed above, all the solutions that are part of the search space of Algorithm 1 are also found in the search space of Algorithm 2, but not all the solutions in the second search space are necessarily found in the first. The search space where Algorithm 2 looks for the solutions matches the search space of the problem.

4.2. Initial population

The population, sized N is generated randomly. First, one vector of each class is selected among all the vectors in that class. Then, the coordinates of the p components of the hyperplanes are set, each of them by generating a random real number in the interval $[-1,1]$. Finally, the p chosen features are randomly selected between all of them. In this way, all the feasible solutions to the problem could be generated without excluding, a priori, any of them. The number of feasible solutions that can be generated with this procedure is infinite, given that, although the subsets of vectors and features are finite, the combinations of real values for the hyperplane coordinates are infinite. It is important to remark that the number of solutions is infinite from a theoretical perspective. In practice, given that the real numbers are coded by a fixed bit depth in computers, the search space would be huge, but not infinite.

4.3. Crossover

The new crossover has been designed in order to efficiently combine the information of two solutions to generate offspring that can inherit the good characteristics of the parents. As the encoding has three different parts, vectors, plane coordinates and features, the combination of two solutions must consider each one of these parts. Once a couple of parents has been selected following the tournament selection based on dominance proposed by Alcaraz et al. (2022), two couples of offspring are generated. Finally, each couple competes for only one place in the

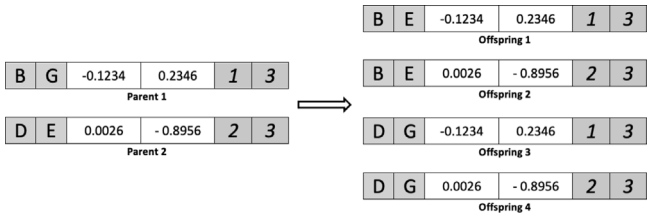


Fig. 9. New crossover example.

Table 5
Objective values for the offspring represented in Fig. 9.

Solution	D	ϵ
Offspring 1	4.1898	4.6093
Offspring 2	0.0043	1.0072
Offspring 3	2.3276	0.9311
Offspring 4	0.0015	0.9999

following population. If one solution dominates the other, that will be the one selected. Otherwise, the winner is randomly selected.

Fig. 9 shows an example of the new crossover operator when applied to two solutions to the problem presented in Example 1. Given two parents, 4 offspring are initially generated. Offspring 1 and Offspring 3 inherit the features array and the plane coordinates from Parent 1 and Offspring 2 and 4 do the same from Parent 2. Finally, Offspring 1 and 2 inherit the first vector from Parent 1 and the second one from Parent 2. Offspring 3 and 4 copy the first vector from parent 2 and the second from Parent 1. In this way, four different offspring, which are also different from the parents are generated. After that, Offspring 1 and 2 would compete for a place in the new population and likewise, Offspring 3 and 4. In this case, if we evaluate the offspring, we have the objective values presented in Table 5. Therefore, as Offspring 1 does not dominate Offspring 2, nor vice versa, the offspring to be copied in the net population would be randomly selected among them. On the other hand, as Offspring 3 dominates Offspring 4, Offspring 3 would be copied in the next population, discarding Offspring 4.

4.4. Mutation

The main purpose of the mutation mechanism is to introduce variability in the population that helps the algorithm avoid being trapped in local optima. A solution undergoes the mutation operation with a given probability of $PMUT$, and if not, it remains unaltered. If the procedure is applied to a given solution, it can alter each one of the three parts of a solution: vectors, plane coordinates and selected features. Firstly, each of the two vectors is replaced by a different one in the same class with a probability of $PMUT_VECTORS$. Similarly, each of the features is replaced by another not present in the solution with a given probability of $PMUT_FEAT$. Finally, each of the plane coordinates can be slightly increased or decreased by a given percentage with a given probability $PMUT_COORD$. This percentage is randomly set for each coordinate in the interval [1,100]. It means that each plane coordinate can be increased or decreased between a 1% and 100%. This interval was set after some preliminary experiments, although varying the limits of the interval did not significantly affect the quality of the solutions.

5. Experimental analysis

We conducted an extensive computational experiment with a dual purpose: firstly, to demonstrate that the new algorithm overcomes the drawbacks of its predecessor and enables the resolution of any instance of the problem, without excluding solutions from the search space, all within highly competitive computation times. Secondly, we compared the efficiency of the new algorithm to the one described in Section 3, showcasing its superiority across a set of well-known instances. The proposed algorithm was calibrated through a design of experiments to identify the best configuration, which was then utilized for comparison with the algorithm from the literature, utilizing in this case the configuration proposed by its authors. All the experiments have been performed on a PC with 2.9 GHz Dual-Core Intel i5 processor, 8 GB of RAM, and the macOS 10.15.6 operating system.

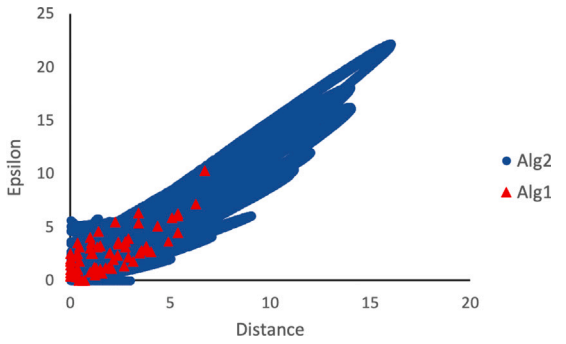


Fig. 10. Feasible solutions in objective space of both algorithms for the example presented in Table 1.

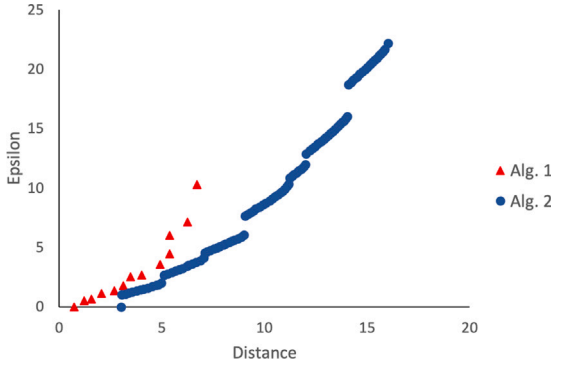


Fig. 11. Graphic comparison of the approximations given by Algorithm 1 and Algorithm 2 for the toy example presented in Table 1.

5.1. Overcoming the drawbacks of Algorithm 1

Our new algorithm, described in the previous section, with the encoding proposed and the operators and procedures that have been designed to work with it, is able to counter the three drawbacks that Algorithm 1 presented: (i) now the search space is not a subset of the original search space; (ii) all the instances of the problem can be solved; (iii) the algorithm is much faster than Algorithm 1.

With regard to the search space, let us use the problem presented in Example 1. As we discussed before, the search space of Algorithm 1 is made up of only 90 solutions, which have been represented, in the objective search space, in Fig. 10. By contrast, the search space of the problem is infinite and it (theoretically) matches the search space of Algorithm 2. To be more precise, the search space of Algorithm 2 would not be infinite, given that real numbers have a finite precision when implemented in a computer. In Fig. 10 we have represented at least 5000 solutions that belong to this second search space.

We have solved this toy example with both algorithms and the frontiers obtained have been represented in Fig. 11. Both algorithms have been run with a computation time limit of 2 s. From the 90 solutions that form the search space of Algorithm 1, only 13 are non-dominated, and those form the frontier that the algorithm gives. As we can observe in the figure, the frontier given by Algorithm 2 dominates

Table 6
No limitation in maximum number of features (p) to be considered by Algorithm 2.

Instance	m	m_1	m_2	n	Max p
Housing	506	249	257	13	n
GC	1000	700	300	24	n
WBC	569	357	212	30	n
Ionosphere	351	126	225	33	n
Arrhythmia	420	183	237	259	n
Mfeat	2000	1800	200	649	n
Colon	62	40	22	2000	n
Leukemia	72	38	34	5327	n
DLBCL	77	19	58	7129	n
Carcinoma	36	17	19	7457	n

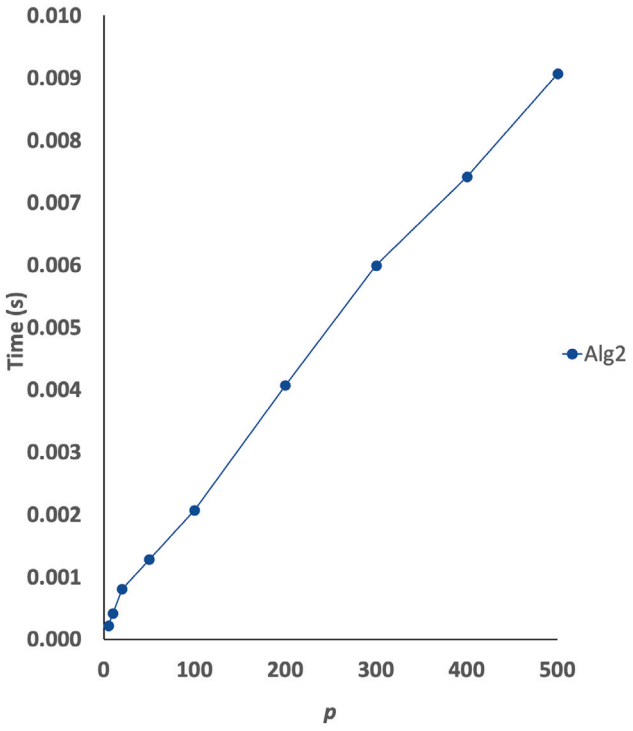


Fig. 12. Average computation times (s) of Algorithm 2 when varying p between 5 and 500.

the 13 solutions that make up the frontier of Algorithm 1. None of the solutions in the frontier given by Algorithm 2 belong to the search space of Algorithm 1. As the search space of Algorithm 2 does not exclude solutions as Algorithm 1, imposed by the type of encoding, the quality of the frontiers given may differ considerably. We will compare the quality of the frontiers given by both algorithms for different datasets in the computational experiment carried out and presented in the next section.

The second drawback presented by Algorithm 1 was related to the instances of the problem that it cannot solve. Algorithm 1 cannot solve many instances of the problem; to be more precise when the number of features to be selected is greater than the maximum number of vectors in both classes, that is when $p > max(m_1, m_2)$. Algorithm 2 does not present this drawback because now the maximum number of features is only limited by the number of dimensions available. This has been represented, for the datasets selected, in Table 6. If we compare Tables 3 and 6 we can realize with these instances that this drawback has been overcome and the algorithm no longer imposes additional restrictions not present in the problem conditions, allowing all possible values of parameter p to be solved in all datasets.

Finally, regarding the speed of Algorithm 1, where the computation time to evaluate one solution dramatically increases as p increases,

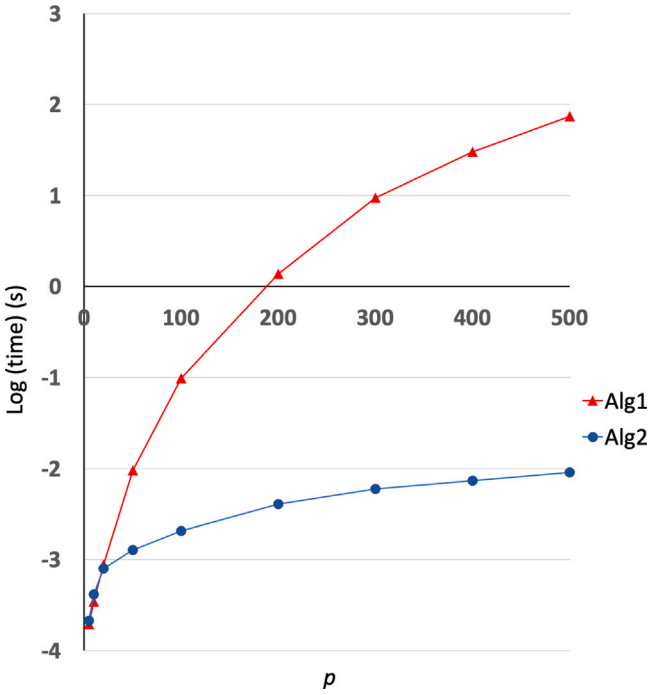


Fig. 13. Comparison of the average computation times (s) of both algorithms when varying p between 5 and 500.

we have measured the computation time that Algorithm 2 needs to evaluate one solution for the set of instances for different p values and these have been presented in Table 7. A first glance to this table gives us the impression that the computation time needed to evaluate one solution increases at the same rate that p increases, regardless of the p value. Varying p from 5 to 10 doubles the time needed and varying p from 1000 to 2000 also duplicates the seconds needed to evaluate one solution. If we now compare the computation times of Algorithm 1 (Table 4) and Algorithm 2 (Table 7) we can evidence that they are quite similar for small values of p. For example, when $p = 5$ Algorithm 1 needs, on average, $2.0E-04$ seconds and Algorithm 2 $2.1E-04$ seconds. However, these differences increase as p increases. If we compare the computation time needed by Algorithm 1 to solve the instance Mfeat with p =n= 649, it needs 210 s to evaluate only one solution in contrast to the 0.055 s that Algorithm 2 employs. We can also observe that the computation times needed by Algorithm 2 are rather small for large p values. This difference in the speed will render a very different efficiency when we compare both algorithms with a given time limit as stopping criterion. In the next section, we compare the quality of the frontiers given by both algorithms through an extensive computational experiment.

The average computation times needed by Algorithm 2 to compute one solution in the set of instances with between 5 and 500 features (in order to allow the comparison with the computation times presented in Fig. 5) has been presented in Fig. 12, where we can observe that the dependence between p and the computational time required approximates to linear. Finally, in order to compare the computation times given by both algorithms when varying p, we have represented them together in Fig. 13. However, in order to visualize them properly, we have represented the logarithms of times instead of times. There, we can observe that although the computation times are similar for small p values, they increase at a very different rate as p increases. The experiments presented above demonstrate that the new algorithm, Algorithm 2, is capable of overcoming all the drawbacks that Algorithm 1 presented. The search space of Algorithm 2 does not exclude solutions, many of which belong to the Pareto front, as it has been demonstrated. Any

Table 7
Computation times (s) to evaluate one solution for Algorithm 2.

Instance	n	p=5	p=10	p=20	p=50	p=100	p=200	p=300
Housing	13	1.9E−04	3.3E−04	–	–	–	–	–
GC	24	3.6E−04	6.3E−04	1.2E−03	–	–	–	–
WBC	30	1.9E−04	3.7E−04	7.2E−04	–	–	–	–
Ionosphere	33	1.5E−04	2.3E−04	5.5E−04	–	–	–	–
Arrhythmia	259	1.6E−04	3.1E−04	4.5E−04	1.1E−03	1.8E−03	3.2E−03	–
Mfeat	649	8.8E−04	2.0E−03	3.7E−03	5.6E−03	8.8E−03	1.8E−02	2.6E−02
Colon	2000	5.3E−05	7.4E−05	1.1E−04	1.9E−04	3.5E−04	6.1E−04	7.8E−04
Leukemia	5327	7.4E−05	1.2E−04	2.0E−04	3.3E−04	4.8E−04	7.5E−04	1.0E−03
DLBCL	7129	4.9E−05	9.0E−05	1.7E−04	3.2E−04	6.4E−04	1.0E−03	1.1E−03
Carcinoma	7457	3.0E−05	4.6E−05	7.6E−05	2.0E−04	3.0E−04	4.9E−04	6.0E−04
Average		2.1E−04	4.2E−04	8.0E−04	1.3E−03	2.1E−03	4.1E−03	6.0E−03

Instance	n	p=400	p=500	p=1000	p=2000	p=5000	p= n
Housing	13	–	–	–	–	–	4.0E−04
GC	24	–	–	–	–	–	1.4E−03
WBC	30	–	–	–	–	–	9.8E−04
Ionosphere	33	–	–	–	–	–	7.9E−04
Arrhythmia	259	–	–	–	–	–	4.8E−03
Mfeat	649	3.3E−02	4.0E−02	–	–	–	5.5E−02
Colon	2000	1.0E−03	1.5E−03	2.6E−03	5.0E−03	–	5.0E−03
Leukemia	5327	1.2E−03	1.5E−03	3.0E−03	5.9E−03	1.4E−02	1.5E−02
DLBCL	7129	1.4E−03	1.7E−03	3.2E−03	6.2E−03	1.5E−02	2.3E−02
Carcinoma	7457	6.9E−04	8.8E−04	1.6E−03	3.0E−03	7.3E−03	1.2E−02
Average		7.4E−03	9.1E−03	2.6E−03	5.0E−03	1.2E−02	1.2E−02

instance of the problem, regardless of its characteristics, can be resolved by Algorithm 2, as opposed to Algorithm 1 and finally, Algorithm 2 allows very large instances to be solved in much shorter computation times while the time required by Algorithm 1 becomes unreasonable for medium or large p values. In the next section, we present the computational experiment carried out in order to analyze the fronts given by Algorithm 2 in contrast to those built by Algorithm 1.

5.2. Metrics for the comparison of algorithms

We have carried out an extensive computational experiment in order to compare the efficiency of Algorithm 1 (Alcaraz et al., 2022) and the new one proposed in this work, Algorithm 2. The comparison is performed through the metrics proposed by the authors of the first study to compare frontier quality. These metrics were chosen to cover the different aspects to be taken into account when analyzing the quality of a frontier, such as cardinality, convergence , spread and distribution (Audet et al., 2021) to calibrate several multi-objective metaheuristics. These metrics are:

- OVNG (Veldhuizen and David, 1999): Overall non-dominated Vector Generation. It gives the number of non-dominated points in the front. Fronts with a higher number of points are desirable.
- C-metric (Zitler and Thiele, 1998). It returns, when comparing two fronts, F_1 and F_2, the fraction of solutions in F_2 that are dominated by one or more solutions in F_1, $C(F_1, F_2)$.
- S-metric or Hypervolume (Zitzler, 1999). It determines the volume (area in the case of two objectives), of the objective space covered by the front. Therefore, a larger area is desired.
- M_3^*-metric (Zitler et al., 2000). It is equal to the distance of the two most distant solutions in the front and, therefore, a higher value is desired. This metric represents the total extent of the front.
- Γ-metric (Custòdio et al., 2011). It calculates the maximum distance between two consecutive points in the front, and consequently, a lower value of this metric is desirable.

We believe that although the metric Γ gives important information about the characteristics of a given frontier, it should not be considered alone, but together with the M_3^*-metric. For instance, we could have two different frontiers, F_1 and F_2 with Γ values of 50 and 100 respectively. Based on these values we could conclude that, as regard the holes, F_1 is much better than F_2. However, if for these frontiers we have M_3^* values of 200 and 1000 respectively, it is easy to calculate that the largest hole of F_1 represents 25% of the total extent and in the case of F_2 it represents only 10% of the total extent. Therefore, taking this into consideration, we could now conclude that, as regards the holes, F_2 performs better than F_1. As a consequence, we propose the μ-metric, calculated as $\Gamma / M_3^* \in [0, 1]$ to determine the quality of a frontier in terms of the relative size, as regard the extent, of the largest hole it presents. Therefore, a lower value of this metric is desirable.

5.3. Calibration of the proposed algorithm

In order to perform an appropriate comparison of the new proposal, Algorithm 2, with the algorithm existing in the literature to solve this bi-objective problem, Algorithm 1, a previous phase to calibrate the new algorithm is necessary.

As described in Section 4, the configuration of the new algorithm depends on five parameters: N (population size), $PMUT$ (mutation probability), $PMUT_FEAT$ (mutation of features probability), $PMUT_VECTORS$ (mutation of vectors probability) and $PMUT_COORD$ (mutation of plane coordinates probability). For the calibration, and based on some preliminary experiments, we have considered two different values for each parameter, which leads to 32 different configurations. These configurations, together with the corresponding parameter values, are given in Table 8.

The algorithm is calibrated on a calibration set of instances. To build such a set, we have generated 9 different random replicates for each one of the original instances in the literature previously described and shown in Table 3. Each replicate has exactly the same number of vectors of each class as the corresponding original. Each vector in a replicate is generated, randomly increasing or decreasing each of the original feature values by up to a 20%. Therefore, the calibration set is made up of a total of 90 instances. The 10 original instances form the evaluation set and will be used later to compare both algorithms.

In order to determine the best configuration, or configurations, of Algorithm 2 we propose a full factorial design of experiments with all combinations of factors (algorithm parameters) and levels as in Yepes-Borrero et al. (2021). Each one of the 90 instances in the calibration set has been solved three independent times with each one of the 32 configurations, establishing $p = 5$ and a maximum computation time of 0.5 min, following the computational analysis to analyze the

Table 8
Calibration phase: configurations of Algorithm 2.

Conf. #	N	PMUT	PMUT_FEAT	PMUT_VECTORS	PMUT_COORD
1	100	0.7	0.2	0.2	0.1
2	100	0.7	0.2	0.2	0.2
3	100	0.7	0.2	0.4	0.1
4	100	0.7	0.2	0.4	0.2
5	100	0.7	0.4	0.2	0.1
6	100	0.7	0.4	0.2	0.2
7	100	0.7	0.4	0.4	0.1
8	100	0.7	0.4	0.4	0.2
9	100	0.9	0.2	0.2	0.1
10	100	0.9	0.2	0.2	0.2
11	100	0.9	0.2	0.4	0.1
12	100	0.9	0.2	0.4	0.2
13	100	0.9	0.4	0.2	0.1
14	100	0.9	0.4	0.2	0.2
15	100	0.9	0.4	0.4	0.1
16	100	0.9	0.4	0.4	0.2
17	200	0.7	0.2	0.2	0.1
18	200	0.7	0.2	0.2	0.2
19	200	0.7	0.2	0.4	0.1
20	200	0.7	0.2	0.4	0.2
21	200	0.7	0.4	0.2	0.1
22	200	0.7	0.4	0.2	0.2
23	200	0.7	0.4	0.4	0.1
24	200	0.7	0.4	0.4	0.2
25	200	0.9	0.2	0.2	0.1
26	200	0.9	0.2	0.2	0.2
27	200	0.9	0.2	0.4	0.1
28	200	0.9	0.2	0.4	0.2
29	200	0.9	0.4	0.2	0.1
30	200	0.9	0.4	0.2	0.2
31	200	0.9	0.4	0.4	0.1
32	200	0.9	0.4	0.4	0.2

performance of Algorithm 1 (Alcaraz et al., 2020). A total of 8640 approximations of the Pareto front are computed as a result.

To compare the different configurations, a set of metrics and a preferred order among them must be established, following the methodology employed by Rodríguez-Ballesteros et al. (2024) to calibrate several multi-objective metaheuristics. The metrics considered in the calibration phase should cover most of the desirable characteristics in a Pareto front, or its approximation, such as cardinality, convergence, spread and distribution. Taking this into account, we have selected three of the metrics described in Section 5.2 that jointly measure all these characteristics: S-metric or hypervolume which is a convergence and distribution measure; C-metric which measures cardinality; and the proposed new metric μ, which combines distribution and spread.

Therefore, these three metrics are the response variables of our statistical analysis. As we have three different values of each one of these metrics per instance, given that we perform three independent runs per instance, we calculate the average for these three. As regards the C-metric, we compute $C(F_2, F_1)$, which is the percentage of solutions in F_1 that are dominated by one or more solutions in F_2, taking F_2 as the front given by Algorithm 2 and F_1 the corresponding frontier given by Algorithm 1. Therefore we need to run Algorithm 1 (with the configuration proposed by the corresponding authors and the same time limit) in the calibration set, also performing 3 runs per instance, which gives a total of 270 approximation fronts to be compared with to determine $C(F_2, F_1)$.

The statistical analysis performed to compare the 32 configurations of Algorithm 2 consists of a two-way analysis of variance (ANOVA) and the response variables are, Hypervolume, $C(F_2, F_1)$ and μ. As may be observed, there is one ANOVA model for each response variable. Let us present, as an example, the ANOVA model for the Hypervolume (HV) response variable. We can consider two distinct factors, the instance to be solved i, and the configuration employed, j. A parametrization of the model can be defined as follows:

$$HV_{ij} = \delta + \alpha_i + \beta_j + \epsilon_{ij}, \quad (1)$$

where $i \in \{1, \ldots, 90I\}$, $j \in \{1, \ldots, 32\}$, $\epsilon_{ij} \in N(0, \sigma^2)$ and are independent. Parameter δ represents the overall mean, parameters α_i represent the main effect of factor i, and parameters β_j represent the main effect of factor j. For the other two response variables, the ANOVA model should be identical.

It is worth noting that all necessary ANOVA hypotheses are checked. Lastly, we employ Tukey's Honest Significant Difference (HSD) test to determine whether or not the means values of the different performance indicators are significantly different from each other. For both the two-way ANOVA and Tukey's HSD test, a confidence level 99% is used.

Due to space considerations, we do not present here the detailed results of these analysis. These results can be found, along with a more detailed description of the ANOVA models and the plots of Tukey's HSD Test, as online materials associated with this work.

However, we summarize the conclusions of the calibration set. With regard to the response variable HV, there are four configurations that statistically perform the best, namely configurations 8, 15, 16 and 7, and there are no significant differences among them. By subsequently applying the next response variable, $C(F2, F1)$ we can conclude that, from these four, the best two are configurations 8 and 7, with no statistical differences among them. Finally, regarding the μ-metric, we discard configuration 7, concluding that the best configuration of this calibration phase is configuration 8. This configuration has the following levels for the configurable parameters: $N = 100$; $PMUT = 0.7$; $PMUT_FEAT = PMUT_VECTORS = 0.4$ and $PMUT_COORD = 0.2$.

It is important to note that configuration 8 would have been the selected configuration regardless of the order of the metrics considered, and this result better supports the conclusion about the election of this configuration as the best among the 32 compared.

The configuration selected has been employed in all the experiments conducted to evaluate the performance of Algorithm 2 and to compare it with the corresponding performance of Algorithm 1, which is described in the next subsection.

5.4. Comparison of algorithms

In order to compare the new algorithm we propose in this work, Algorithm 2, with the only one that currently exists to solve this problem, Algorithm 1, we have used the configuration of Algorithm 2 selected in the calibration phase in all the experiments. With regard to Algorithm 1, we have employed the configuration proposed by the authors when solving the same set of instances that we are employing in this comparison. The configuration of Algorithm 1, determined by 5 parameters is: $N = 100$; $PCROSSS_BEST = 0.8$; $PMUT = 0.7$; $PMUT_FEAT = PMUT_VECTORS = 0.4$

First, we have compared both algorithms solving the datasets used in the computational experiment to analyze the performance of Algorithm 1 (Alcaraz et al., 2022). Hereinafter, we shall refer to this set, which is made up of 10 instances, as the evaluation set. We have solved the instances with two different p values, $p = 5$ (the only one considered in the computational study referred to previously) and the maximum value of p that Algorithm 1 allows for each instance, which is represented in Table 3. The computation time limit for both algorithms has been set, in minutes, and equal to $p * 10^{-1}$. Each run has been carried out three independent times to reduce the randomness of the algorithms comparing the results, and the averages of the different metrics have been computed.

Table 9 shows the comparison for the selected instances when $p = 5$ and the computation time limit is set to 0.5 min. Before analyzing the results given in this table, it is interesting to point out that the speed of both algorithms for evaluating one solution when $p = 5$ is, on average, rather similar, $2.0E-04$ s for Algorithm 1 and $2.1E-04$ s for Algorithm 2 (Tables 4 and 7). Therefore, in this comparison, the better efficiency of an algorithm versus the other cannot be blamed on the speed factor.

Table 9
Comparison of Algorithm 1 and Algorithm 2 through several metrics, $p = 5$.

Instance	p	t	OVNG		C		M_3		Γ		S		μ	
			Alg.1	Alg.2	C(F_1, F_2)	C(F_2, F_1)	Alg.1	Alg.2	Alg.1	Alg.2	Alg.1	Alg.2	Alg.1	Alg.2
Housing	5	0.5	91.3	100.0	60%	72%	577.3	676.3	211.8	26.9	1507.0	1549.2	37%	4%
GC	5	0.5	7.0	100.0	59%	62%	1363.1	1650.9	691.7	41.6	3970.2	4238.5	52%	3%
WBC	5	0.5	100.0	100.0	3%	75%	463.3	760.2	23.0	18.3	927.4	2098.4	5%	2%
Ionosphere	5	0.5	91.7	100.0	30%	98%	371.8	498.4	139.8	14.8	913.9	1369.4	38%	3%
Colon	5	0.5	100.0	100.0	44%	17%	86.2	107.9	2.7	3.5	182.7	274.1	3%	3%
Leukemia	5	0.5	100.0	100.0	31%	41%	103.3	126.3	7.8	6.7	265.6	370.3	8%	5%
DLBCL	5	0.5	100.0	100.0	16%	69%	73.8	88.2	2.2	3.1	176.8	244.3	3%	4%
Carcinoma	5	0.5	100.0	100.0	15%	69%	34.1	58.3	3.7	4.4	86.8	192.8	11%	7%
Arrhythmia	5	0.5	100.0	100.0	20%	83%	615.4	782.2	142.3	26.2	1263.5	1855.8	23%	3%
Mfeat	5	0.5	51.0	100.0	30%	97%	2159.0	2929.4	449.6	495.1	5697.4	10 563.1	20%	17%

Table 10
Comparison of metrics, p is the maximum value that Algorithm 1 allows.

Instance	p	t	OVNG		C		M_3		Γ		S		μ	
			Alg.1	Alg.2	C(F_1, F_2)	C(F_2, F_1)	Alg.1	Alg.2	Alg.1	Alg.2	Alg.1	Alg.2	Alg.1	Alg.2
Housing	13	1.3	10.7	100.0	59%	0%	247.4	999.5	143.9	28.3	725.3	3101.1	51%	3%
GC	24	2.4	5.7	100.0	45%	0%	344.3	2604.0	270.1	99.5	1125.5	10 231.6	67%	4%
WBC	30	3.0	0.0	100.0	–	–	–	1039.6	–	35.8	–	3822.4	–	3%
Ionosphere	33	3.3	3.3	100.0	56%	0%	25.0	979.4	18.9	25.5	98.6	4288.1	85%	3%
Arrhythmia	237	23.7	100.0	100.0	25%	0%	55.9	260.9	2.9	8.4	85.2	1395.2	5%	3%
Mfeat	649	64.9	100.0	100.0	8%	68%	178.7	218.2	49.8	9.3	620.1	926.4	23%	4%
Colon	40	4.0	93.7	100.0	3%	90%	71.9	263.4	5.0	26.9	157.3	1634.9	7%	10%
Leukemia	38	3.8	100.0	100.0	9%	89%	39.6	80.3	2.2	9.3	114.4	343.0	5%	12%
DLBCL	58	5.8	0.0	100.0	–	–	–	769.4	–	23.2	–	1789.3	–	3%
Carcinoma	19	1.9	1.0	100.0	2%	0%	–	6303.1	–	310.9	–	47 862.4	–	5%

Analyzing the metrics, first, the number of solutions on the front given by Algorithm 2 is always 100 and in the case of Algorithm 1 it varies between 7 and 100. It is worth remembering that, as the population size employed by both algorithms is $N = 100$, the maximum number of points (solutions) in a frontier given by any of the algorithms is limited to 100. With regard to the metric C, we can observe that, except for one instance (Colon) the percentage of solutions of one front that are dominated by solutions of the other is always better for Algorithm 2. In the most extreme case, the WBC instance, only 3% of solutions in frontier F_2 are dominated by solutions in frontier F_1. By contrast, 75% of the points in F_1 are dominated by points in F_2. M_3, that is, the total extent of the front is in all cases larger for Algorithm 2, which represents a better performance in this sense. A similar comparison can be made for the hyperarea covered by the frontiers, that in all cases it is better for Algorithm 2. As regard the size of the holes on the frontier, there are only three of the ten instances in which it is worse in Algorithm 2, but if we relativize it to the total extent, it is always better for Algorithm 2 except for one instance (WBC) in which they are rather similar. Therefore, the results in Table 9 show that the frontiers given by Algorithm 2 have a higher number of solutions, cover a higher fraction of solutions of the other, cover a higher area, have a greater distance between the extreme points and smaller holes, and therefore we can conclude that the quality of these frontiers is superior to that of its predecessor. For that reason, this experiment demonstrates the superiority of Algorithm 2.

Table 10 shows the different metrics calculated for the frontiers given by both algorithms, which are approximations of the Pareto front, when solving the set of instances. The value of p is set, for each instance, as the maximum value that Algorithm 1 is capable of solving for the corresponding instance. In all the cases, Algorithm 2 obtains the maximum number of points in the frontier, 100, while Algorithm 1 only reaches that number in three of the ten instances. Moreover, in two cases, Algorithm 1 does not obtain any point and in four instances, the number of points in the frontier is really small. However, for the instances Housing, GC, Ionosphere and Carcinoma, although the number of points of the frontier given by Algorithm 1 are very reduced, these points are non-dominated by points of frontier given by Algorithm 2 (C(F_2, F_1)=0%). This also happens in the case of Arrhythmia, although for this instance both algorithms obtain frontiers with 100 points. In three of the instances, Mfeat, Colon and Leukemia, a large proportion of points of F_1 are dominated by points of F_2. As regard the extent (M_3) and the hyperarea (S) covered by the frontiers, these metrics are always much better for Algorithm 2. If we look at the relative size of the holes, it is always much better for Algorithm 2, except for two instances, Colon and Leukemia, although in these cases the differences between both are rather small, compared with the other instances. Summarizing the results in this table, it is remarkable that in three of the cases, Algorithm 1 does not even have a frontier. In the rest of the cases, most of the metrics corroborate once more that the frontiers given by Algorithm 2 can be considered as a much better quality than those of Algorithm 1.

In the previous experiments, the only instances that allow the comparison in the performance of the algorithms for large values of p ($200 \leq p$) are Arrhythmia and Mfeat, that allow a maximum number of 237 and 649 respectively. In order to carry out a more thorough comparison, we have selected one more instance from the UCI repository, Madelon, which presents 2000 vectors (1000 of each class) and 500 dimensions and therefore allows both algorithms to be compared for several large values of p. Table 11 shows the comparison for large values of p ($200 \leq p \leq 500$) for the Madelon and Mfeat instances. First, we can observe the very large differences when comparing the number of points in the corresponding frontiers. The number of points vary, for Algorithm 1 between 1 and 74, on average, although in six of the eight cases, it is a lot lower, below 6, being only 1 point in two cases. However, most of these points are not dominated by points given by Algorithm 2. We can also observe that, for Algorithm 1, OVNG reduces as p increases. This metric for Algorithm 2 remains much more stable with much better values, varying between 80 and 100, on average. As in the previous cases, the extent, hyperarea and relative size of the holes, are always much better for Algorithm 2. This table evidences that the differences between the quality of the frontiers given by both algorithms are even larger for large values of p. When the number of features increases, the frontiers given by Algorithm 1 are very poor in terms of number of solutions. Moreover, the rest of the metrics also show that the characteristics of the frontiers given by Algorithm 1 worsen as p increases and this is not evidenced for Algorithm 2.

Table 11
Instances with large number of dimensions. Comparison of metrics for several large values of p.

Instance	p	t	OVNG		C		M_3		Γ		S		μ	
			Alg.1	Alg.2	C(F_1, F_2)	C(F_2, F_1)	Alg.1	Alg.2	Alg.1	Alg.2	Alg.1	Alg.2	Alg.1	Alg.2
Madelon	200	20	74.3	83.3	15%	34%	692.9	1606.7	85.8	69.6	224.0	1634.4	12%	4%
Madelon	300	30	47.7	100.0	16%	18%	626.2	1730.4	120.2	88.6	201.4	1859.9	19%	5%
Madelon	400	40	1.0	100.0	0%	0%	–	1790.4	–	90.4	–	1964.9	–	5%
Madelon	500	50	1.0	100.0	0%	0%	–	2554.4	–	70.0	–	4191.2	–	3%
Mfeat	200	20	5.7	80.0	33%	39%	1386.0	5772.1	1124.7	1064.2	2111.9	29 562.6	81%	19%
Mfeat	300	30	2.7	93.3	32%	0%	7.5	4301.7	6.6	568.3	7.5	12 817.6	90%	13%
Mfeat	400	40	2.7	88.7	28%	0%	4.4	6368.6	3.5	1143.9	5.5	32 042.0	81%	19%
Mfeat	500	50	2.0	84.3	40%	0%	8.0	6526.3	8.0	1557.5	9.2	34 399.4	100%	23%

The results presented in the three previous tables indicate that, in general, Algorithm 2 gives much more consistent frontiers, with a larger extent, a larger hyperarea covered and smaller relative holes than Algorithm 1. Therefore, we can conclude, considering an overview of all the experiments and metrics altogether, that Algorithm 2 performs much better than Algorithm 1 when it comes to generating an approximation of the Pareto front of the problem considered.

6. Conclusions and final remarks

In this paper, we deal with the soft margin SVM with the selection of a given number of features. Although it is a hard multi-objective problem, most of the techniques developed to solve it build a unique objective function and they do not ensure the complete set of non-dominated solutions. As far as we know, there is only one metaheuristic to deal with the problem from a pure multi-objective perspective, and, although the results presented by the authors demonstrated its efficiency when compared with an exact technique, these results have been greatly improved by the new metaheuristic proposed. It is based on the same paradigm as its predecessor but has been completely redesigned. A new encoding and efficient procedures designed to work with it allow the main limitations of its predecessor to be countered: (i) reduce the search space, excluding many solutions ; (ii) not allow solving some instances for some values of p and (iii) need an important computational effort to evaluate the solutions, which makes the technique slow. The computational experiments show that the new technique outperforms the results given by its predecessor. This important improvement in the results cannot be attributed only to the better speed of the new algorithm, but also to the considered search space, given that the search space of the new technique coincides with the search space of the problem. The experiments have been carried out in a given set of diverse instances and several metrics, that cover different desired characteristics in a frontier, have been computed. In this sense, we propose a new metric, that combines two well-known metrics, to calculate the relative size of the holes of a front with regard to the complete extent.

This work can be used as a base for several interesting and promising research lines. First, some other metaheuristics could be developed to solve the problem and could be compared with the one proposed here. Secondly, some other interesting and practical variants of the problem could be studied and solved through metaheuristics such as, for example, the multiple-class SVM when we have more than two vector classes; the weighted SVM that is used in imbalanced instances and enables higher weight to be assigned to the less represented classes; the ν-SVM where the number of support vectors and the margin width can be controlled through a parameter. Perhaps some other additional objectives could be considered such as, for example, different objectives related to the features: to minimize the number of selected features or, if there is a cost associated to them, minimizing the total cost of the selected features. In these cases, if one or more of those objectives are added to the two classical ones, a specific metaheuristic to deal with multi-objective problems with more than two objectives could be designed.

CRediT authorship contribution statement

Javier Alcaraz: Conceptualization, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Writing – original draft, Writing – review & editing.

Data availability

Data will be made available on request.

Acknowledgments

The author thanks the grants PID2022-136383NB-I00 funded by MICIN/AEI/ 10.13039/501100011033 and by ERDF A way of making Europe; and PROMETEO/2021/063 funded by the autonomous government of the Valencian Region, Spain.

Appendix A. Supplementary material

Supplementary material associated with this article, which includes the results of the two-way ANOVA and Tukey's Honest Significant Difference (HSD) test, can be found in the online version at https://doi.org/10.1016/j.cor.2024.106821.

References

Al-Tashi, Q., Abdulkadir, S.J., Rais, H.M., Mirjalili, S., Alhussian, H., 2020. Approaches to multi-objective feature selection: A systematic literature review. IEEE Access 8, 125076–125096.

Aladeemy, M., Tutun, S., Khasawneh, M.T., 2017. A new hybrid approach for feature selection and support vector machine model selection based on self-adaptive cohort intelligence. Expert Syst. Appl. 88, 118–131.

Alcaraz, J., Labbé, M., Landete, M., 2022. Support vector machine with feature selection: A multiobjective approach. Expert Syst. Appl. 204.

Alcaraz, J., Landete, M., Monge, J., Sainz-Pardo, J.L., 2020. Multi-objective evolutionary algorithms for a reliability location problem. European J. Oper. Res. 283, 83–93.

Amini, N., Mahdavi, M., Choubdar, H., Abedini, A., Shalbaf, A., Lashgari, R., 2023. Automated prediction of COVID-19 mortality outcome using clinical and laboratory data based on hierarchical feature selection and random forest classifier. Comput. Methods Biomech. Biomed. Eng. 26 (2), 160–173.

Audet, C., Bigeon, J., Cartier, D., Digabel, S.L., Salomon, L., 2021. Performance indicators in multiobjective optimization. European J. Oper. Res. 292, 397–422.

Aytug, H., 2015. Feature selection for support vector machines using generalized Benders decomposition. European J. Oper. Res. 244 (1), 210–218.

Bouraoui, A., Jamoussi, S., Benayed, Y., 2018. A multi-objective genetic algorithm for simultaneous model and feature selection for support vector machines. Artif. Intell. Rev. 50, 261–281.

Bradley, P.S., Mangasarian, O.L., 1998. Feature selection via concave minimization and support vector machines. In: ICML, Vol. 98. Citeseer, pp. 82–90.

Candelieri, A., Giordani, I., Archetti, F., Barkalov, K., Meyerov, I., Polovinkin, A., Sysoyev, A., Zolotykh, N., 2019. Tuning hyperparameters of a SVM-based water demand forecasting system through parallel global optimization. Comput. Oper. Res. 106, 202–209.

Carrizosa, E., Martin-Barragan, B., Morales, D.R., 2009. Binarized support vector machines. INFORMS J. Comput. 22, 154–167.

Carrizosa, E., Martin-Barragan, B., Morales, D.R., 2014. A nested heuristic for parameter tuning in support vector machines. Comput. Oper. Res. 43, 328–334.

Cervantes, J., Garcia-Lamont, F., Rodríguez-Mazahua, L., Lopez, A., 2020. A comprehensive survey on support vector machine classification: Applications, challenges and trends. Neurocomputing 408, 189–215.

Cortes, C., Vapnik, V., 1995. Support-vector networks. Mach. Learn. 20 (3), 273–297.

Custòdio, A.L., Madeira, J., Vaz, A., Vicente, L., 2011. Direct multisearch for multiobjective optimization. SIAM J. Optim. 21, 1109–1140.

Deb, K., Pratap, A., Agarwal, S., Meyarivan, T., 2002. A fast and elitist multi-objective genetic algorithm: NSGA-II. IEEE Trans. Evol. Comput. 6, 182–197.

Ding, H., Huang, Q., Rodriguez, D., 2023. Modified Locust Swarm optimizer for oral cancer diagnosis. Biomed. Signal Process. Control 83, 104645.

Dudzik, W., Nalepa, J., Kawulok, M., 2021. Evolving data-adaptive support vector machines for binary classification. Knowl.-Based Syst. 227, 107221.

Faris, H., Hassonah, M.A., Al-Zoubi, A., Mirjalili, S., Aljarah, I., 2018. A multi-verse optimizer approach for feature selection and optimizing SVM parameters based on a robust system architecture. Neural Comput. Appl. 30, 2355–2369.

García-Pedrajas, N., de Haro-García, A., Pérez-Rodríguez, J., 2014. A scalable memetic algorithm for simultaneous instance and feature selection. Evolut. Comput. 22, 1–45.

Gaudioso, M., Gorgone, E., Labbé, M., Rodríguez-Chía, A., 2017. Lagrangian relaxation for SVM feature selection. Comput. Oper. Res. 87, 137–145.

Gauthama Raman, M., Somu, N., Kirthivasan, K., Liscano, R., Shankar Sriram, V., 2017. An efficient intrusion detection system based on hypergraph - Genetic algorithm for parameter optimization and feature selection in support vector machine. Knowl.-Based Syst. 134, 1–12.

Golub, T., Slonim, D., Tamayo, P., Huard, C., Gaasenbeek, M., Merisov, J., Coller, H., Loh, M., Downing, J., Caligiuri, M., Bloomfeld, C., Lander, E., 1999. Molecular classification of cancer: Class discovery and class prediction by gene expression monitoring. Science 286, 531–537.

Gu, Z., Zhang, Z., Sun, J., Li, B., 2017. Robust image recognition by L1-norm twin-projection support vector machine. Neurocomputing 223, 1–11.

Guyon, I., Weston, J., Barnhill, S., Vapnik, V., 2002. Gene selection for cancer classiffication using support vector machines. Mach. Learn. 46, 389–422.

Huang, C.-L., Wang, C.-J., 2006. A GA-based feature selection and parameters optimization for support vector machines. Expert Syst. Appl. 31 (2), 231–240.

Huang, X., Zhang, L., Fangzhang, B.W., Zhang, Z., 2018. Feature clustering based support vector machine recursive feature elimination for gene selection. Appl. Intell. 48, 594–607.

Jiao, R., Xue, B., Zhang, M., 2022. Benefiting from single-objective feature selection to multiobjective feature selection: A multiform approach. IEEE Trans. Cybern. 1–14.

Kelly, M., Longjohn, R., Nottingham, K., 1987. The UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences, URL http://archive.ics.uci.edu/ml.

Labbé, M., Martínez-Merino, L., Rodríguez-Chía, A.M., 2019. Mixed integer linear programming for feature selection in support vector machine. Discrete Appl. Math. 261, 276–304.

Li, L., Xuan, M., Lin, Q., Jiang, M., Ming, Z., Tan, K.C., 2023. An evolutionary multitasking algorithm with multiple filtering for high-dimensional feature selection. IEEE Trans. Evol. Comput. 27 (4), 802–816.

Lin, S.-W., Lee, Z.-J., Chen, S.-C., Tseng, T.-Y., 2008a. Parameter determination of support vector machine and feature selection using simulated annealing approach. Appl. Soft Comput. 8 (4), 1505–1512, Soft Computing for Dynamic Data Mining.

Lin, S.-W., Ying, K.-C., Chen, S.-C., Lee, Z.-J., 2008b. Particle swarm optimization for parameter determination and feature selection of support vector machines. Expert Syst. Appl. 35 (4), 1817–1824.

Luaces, O., Taboada, F., Albaiceta, G.M., Domínguez, L.A., Enríquez, P., Bahamonde, A., 2009. Predicting the probability of survival in intensive care unit patients from a small number of variables and training examples. Artif. Intell. Med. 45 (1), 63–76.

Maldonado, S., Pérez, J., Weber, R., Labbé, M., 2014. Feature selection for support vector machines via mixed integer linear programming. Inform. Sci. 279, 163–175.

Mavrotas, G., Florios, K., 2013. An improved version of the augmented ϵ-constraint method (AUGMECON2) for finding the exact pareto set in multi-objective integer programming problems. Appl. Math. Comput. 219, 9652–9669.

Middleton, M., Närhi, P., Arkimaa, H., Hyvönen, E., Kuosmanen, V., Treitz, P., Sutinen, R., 2012. Ordination and hyperspectral remote sensing approach to classify peatland biotopes along soil moisture and fertility gradients. Remote Sens. Environ. 124, 596–609.

Notterman, D., Alon, U., Sierk, Levine, A., 2001. Transcriptional gene expression profiles of colorectal adenoma, adenocarcinoma, and normal tissue examined by oligonucleotide arrays. Cancer Res. 61, 3124–3130.

Pai, P.-F., Hsu, M.-F., Wang, M.-C., 2011. A support vector machine-based model for detecting top management fraud. Knowl.-Based Syst. 24 (2), 314–321.

Rodríguez-Ballesteros, S., Alcaraz, J., Anton-Sanchez, L., 2024. Metaheuristics for the bi-objective resource-constrained project scheduling problem with time-dependent resource costs: An experimental comparison. Comput. Oper. Res. 163.

Shipp, M., Ross, K., Tamayo, P., Weng, A.P., Kutok, J., Aguiar, R., Gaasenbeek, M., Angelo, M., Reich, M., Pinkus, G., Ray, T., Koval, M., Last, K., Norton, A., Lister, T., Mesirov, J., Neuberg, D., Lander, E., Aster, J., T.R, G., 2002. Diffuse large b-cell lymphoma outcome prediction by gene-expression profiling and supervised machine learning. Nature Med. 8, 68–74.

Tay, D., Poh, C.L., Goh, C., Kitney, R.I., 2014. A biological continuum based approach for efficient clinical classification. J. Biomed. Inform. 47, 28–38.

Vapnik, V., 1995. The Nature of Statistical Learning Theory. Springer science + business media.

Veldhuizen, V., David, A., 1999. Multiobjective Evolutionary Algorithms: Classifications, Analyses, and New Innovations. Technical Report, School of Engineering of the Air Force Institute of Technology, Dayton, Ohio.

Wang, P., Xue, B., Liang, J., Zhang, M., 2023. Feature selection using diversity-based multi-objective binary differential evolution. Inform. Sci. 626, 586–606.

Wei, W., Xuan, M., Li, L., Lin, Q., Ming, Z., Coello Coello, C.A., 2023. Multiobjective optimization algorithm with dynamic operator selection for feature selection in high-dimensional classification. Appl. Soft Comput. 143, 110360.

Yepes-Borrero, J.C., Perea, F., Ruiz, R., Villa, F., 2021. Bi-objective parallel machine scheduling with additional resources during setups. European J. Oper. Res. 292 (2), 443–455.

Zhang, Z., Chow, T.W., 2012. Maximum margin multisurface support tensor machines with application to image classification and segmentation. Expert Syst. Appl. 39 (1), 849–860.

Zhao, M., Fu, C., Ji, L., Tang, K., Zhou, M., 2011. Feature selection and parameter optimization for support vector machines: A new approach based on genetic algorithm with feature chromosomes. Expert Syst. Appl. 38 (5), 5197–5204.

Zitler, E., Deb, K., Thiele, L., 2000. Comparison of multiobjective evolutionary algorithms: Empirical results. Evolut. Comput. 8, 173–195.

Zitler, E., Thiele, L., 1998. Multiobjective optimization using evolutionary algorithms—A comparative case study. In: Eiben, A., et al. (Eds.), Parallel Problem Solving from Nature. Springer, pp. 292—-301.

Zitzler, E., 1999. Evolutionary algorithms for multiobjective optimization: Methods and applications (Ph.D. thesis). Swiss Federal Institute of Technology.